



UPO UNIVERSITÀ DEL PIEMONTE ORIENTALE

Università Del Piemonte Orientale
"Amedeo Avogadro"

Polo Formativo di Vercelli

Corso di laurea "Informatica" a.a 2024/2025

Progetto **MobiShare** per il corso di:

PROGETTAZIONE E IMPLEMENTAZIONE DI SISTEMI SOFTWARE IN RETE (MF0244)

Gruppo 12

Baselice Asia (20049304)
Bistoncini Sara (20050074)
Duratti Matteo (20050673)
Papa Marco Yuri (20051241)

Indice

1. Specifica.....	3
1.1 Scopo e contesto.....	3
1.2 Requisiti funzionali.....	3
1.3 Requisiti non funzionali e scelte tecnologiche.....	6
Architettura generale.....	6
Tech Stack.....	6
Emulatori e sistemi utilizzati.....	6
Rappresentazione interazioni backend.....	6
1.4 Dominio.....	7
Diagramma delle classi del dominio.....	7
2. Progettazione.....	8
2.1 Architettura generale.....	8
2.2 Componenti principali.....	8
2.3 Dominio applicativo e modello dati.....	8
2.4 Architettura logica e pattern MVC.....	9
2.5 Tecnologie e integrazioni.....	9
2.6 Struttura del database (Firestore).....	10
Motivazioni della scelta di un database NoSQL.....	10
Struttura dei Documenti.....	11
Documento: User.....	11
Documento: Mezzi.....	11
Documento: Parcheggi.....	12
Documento: Rides.....	12
Documento: danni.....	13
Documento: Ricariche.....	13
Documento: Tariffe.....	13
Documento: Shop.....	14
Documento: Redemption.....	14
Diagramma UML.....	14
2.7 Diagrammi di sequenza.....	15
3 Comunicazione IoT e protocollo MQTT.....	16
3.1 Funzionamento MQTT.....	16
3.2 Emulatori.....	16
Philips Hue Emulator.....	16
MobiShare Emulator.....	17
3.2 Topic Principali.....	17
3.4 Flusso Comunicazione.....	17
3.5 Gestione Stati Mezzi Elettrici.....	17
4.0 Arduino (ESP8266).....	18
4.1 Prototipo Arduino con Comunicazione MQTT.....	18
4.2 Configurazione Hardware.....	18
4.3 Funzionalità Implementate.....	18
4.4 Integrazione con il Sistema MQTT.....	18
5.0 Diagrammi di stato.....	19

<<NB: La relazione per il “modulo” di Applicazioni Intelligenti é stata realizzata separatamente, in questa verranno descritti unicamente gli elementi interessati al corso di PISSIR.>>

Specifica

1.1 Scopo e contesto

Il progetto **Mobishare** nasce per gestire un servizio di mobilità condivisa nella città di *Cleanair*, permettendo ai cittadini di spostarsi in modo sostenibile e veloce tramite tre tipi di veicoli:

- **Biciclette muscolari**
- **Biciclette elettriche**
- **Monopattini elettrici**

Ogni mezzo è dotato di **spie luminose** che indicano lo stato (verde per disponibile, rossa per non prelevabile e blu per quelli in uso), mentre per quelli elettrici, viene monitorato anche il livello di **batteria** (verde-arancione-rosso).

Mobishare prevede due ruoli principali:

- **Utente:** può noleggiare mezzi tramite un credito virtuale ricaricabile (Stripe API di test), lasciare feedback, accumulare punti fedeltà (per corse non assistite), visualizzare il profilo e lo storico corse.
- **Gestore:** gestisce parcheggi e veicoli, verifica guasti, batterie scariche, sospensioni utenti e comandi IoT.

Il sistema integra così in un'unica piattaforma la gestione di **utenti**, **mezzi** e **credito**, offrendo un servizio completo di sharing cittadino.

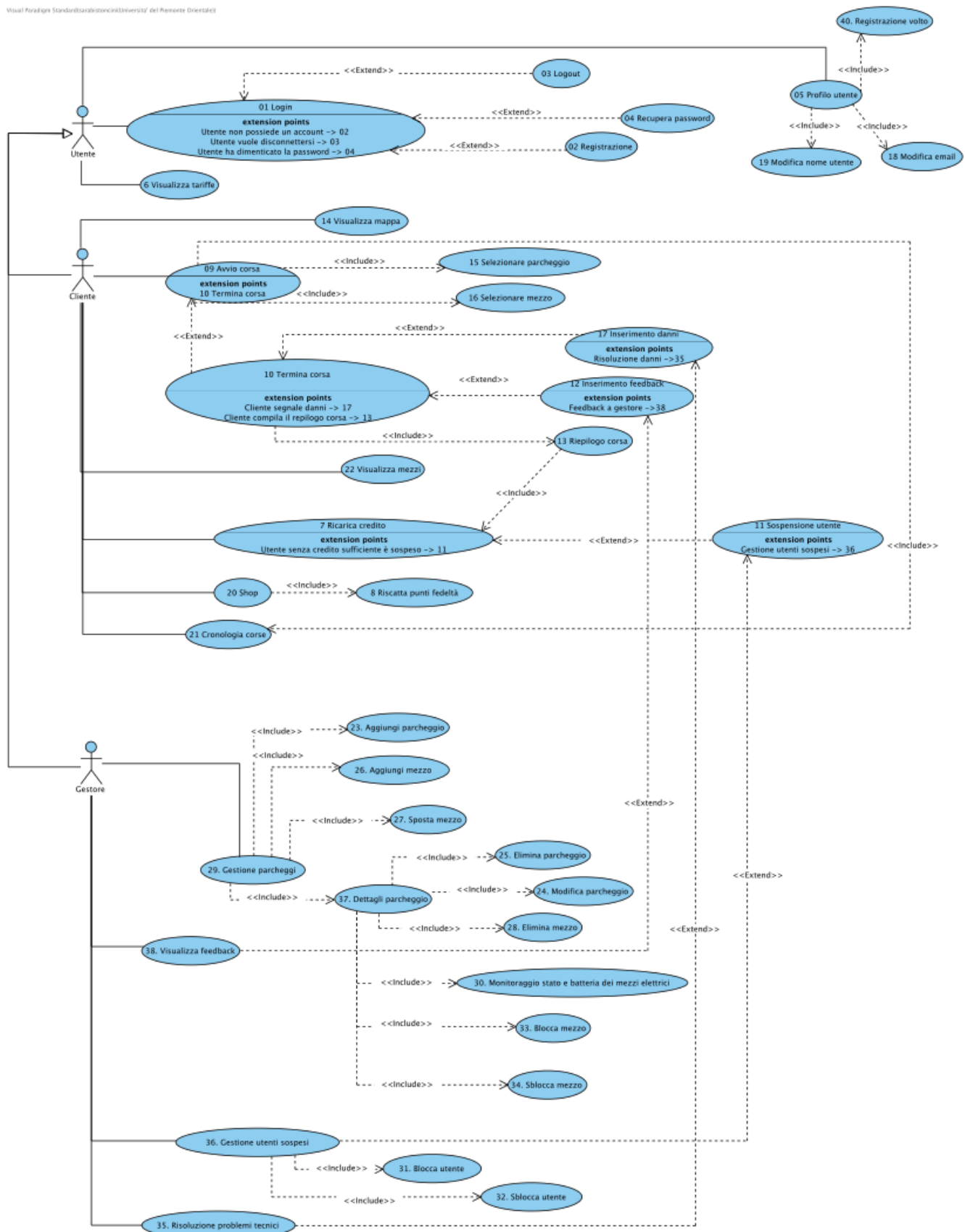
1.2 Requisiti funzionali

1. Login
2. Registrazione
3. Logout
4. Recupera password
5. Profilo utente (dati personali, storico corse, punti fedeltà)
6. Visualizzazione tariffe
7. Ricarica credito (Stripe)
8. Riscatto punti fedeltà (shop → credito)
9. Avvio corsa
10. Termine corsa
11. Sospensione utente
12. Inserimento feedback
13. Riepilogo corsa
14. Visualizza mappa
15. Selezionare parcheggio
16. Selezionare mezzo
17. Inserimento danni
18. Modifica email
19. Modifica nome utente

20. Shop
21. Cronologia corse
22. Visualizza mezzi
23. Aggiungi parcheggio
24. Modifica parcheggio
25. Elimina parcheggio
26. Aggiungi mezzo
27. Sposta mezzo
28. Elimina mezzo
29. Gestione parcheggi
30. Monitoraggio stato e batteria dei mezzi elettrici
31. Bloccare utente
32. Sbloccare utente
33. Bloccare mezzo
34. Sbloccare mezzo
35. Risoluzione problemi tecnici
36. Gestione utenti sospesi
37. Dettagli parcheggio
38. Visualizza feedback
39. Registrazione volto

Diagramma dei casi d'uso

Visual Paradigm Standard (sabbatini@universita' del Piemonte Orientale)



1.3 Requisiti non funzionali e scelte tecnologiche

Architettura generale

- Architettura a componenti (microservizi leggeri)
- Accesso al database solo tramite backend
- Comunicazione real-time con Socket.IO e MQTT

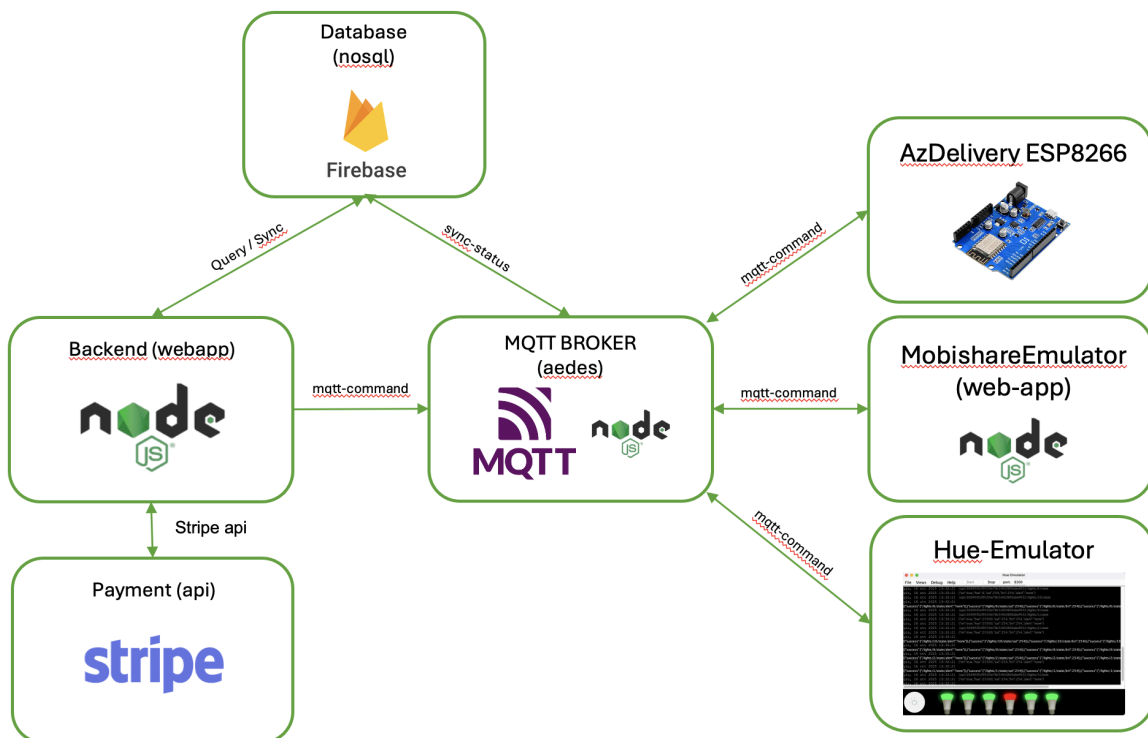
Tech Stack

- **Backend:** Node.js + Express
- **Frontend:** EJS + Bootstrap
- **Database:** Google Firebase Firestore (NoSQL)
- **IoT:** Broker Aedes (MQTT) + emulatori Hue / Mobishare Emulator
- **Pagamenti:** Stripe (API di test)
- **Mappe:** OpenStreetMap
- **Hardware di test:** Arduino AzDelivery D1 con ESP8266

Emulatori e sistemi utilizzati

- Hue-Emulator (mostrato a lezione).
- Mobishare-Emulator: web app emulator realizzata da noi per imitare la funzionalità di hue-emulator ma aggiungendo funzionalità (esempio mostrare più dati, etichette, aggiungere e togliere “lampadine” etc etc).
- Arduino “AzDelivery D1” con modulo esp8266 per la connessione wireless (usato per comunicare con il broker mqtt), l'arduino verrà usato come testing.

Rappresentazione struttura e interazioni nel backend



1.4 Dominio

Le entità principali individuate nel dominio dell'applicazione *Mobishare* sono:

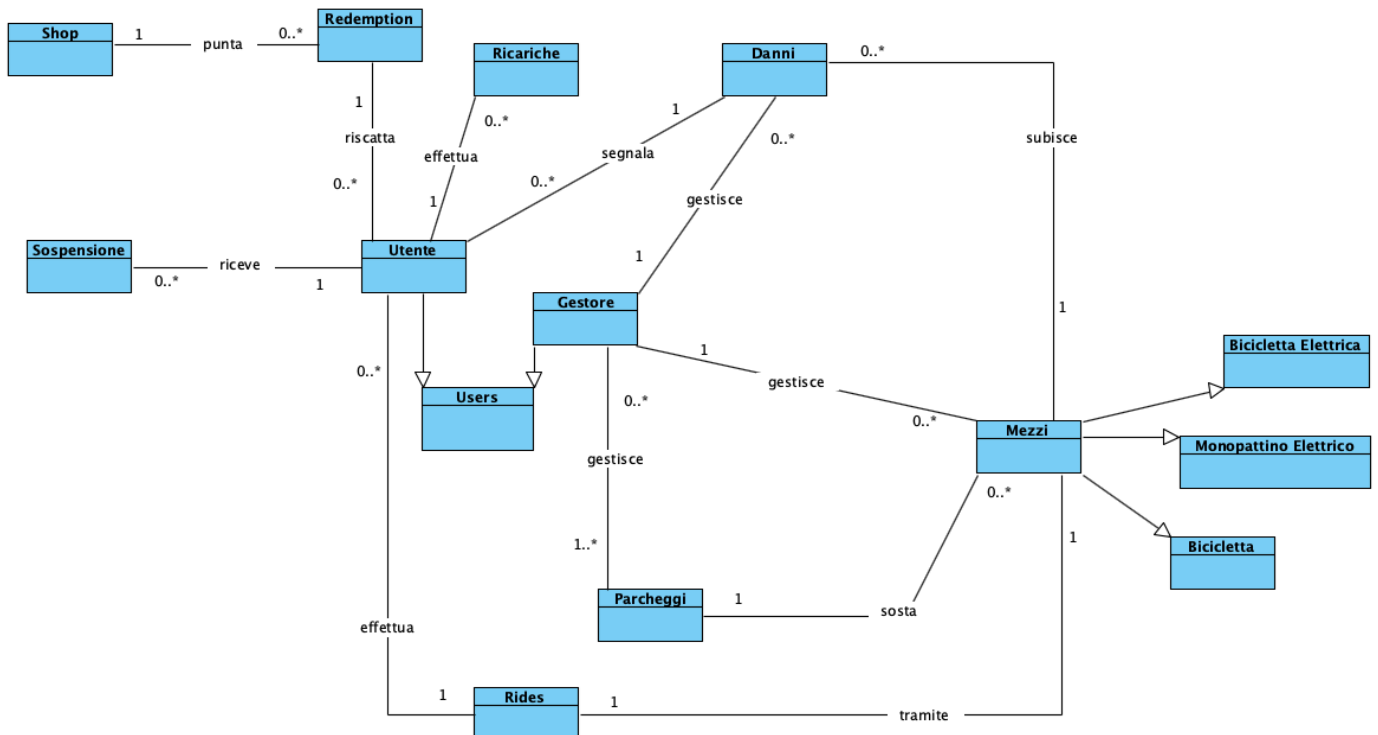
- Utente
- Gestore
- Parcheggio
- Mezzo
- Rides
- Danno
- Ricariche
- Shop
- Redemption
- Sospensione

Il dominio è articolato in due aree principali:

- **Area utente:** comprende la gestione del profilo, del credito, dei punti fedeltà, delle corse e delle eventuali segnalazioni di danni.
- **Area gestore:** comprende il monitoraggio dei mezzi, dei parcheggi, delle segnalazioni dagli utenti, la gestione delle sospensioni e la manutenzione della flotta.

Diagramma delle classi del dominio

Rappresenta in dettaglio le associazioni tra le entità principali e fornisce una visione completa delle interazioni necessarie per la gestione del servizio *Mobishare*.



2. Progettazione

2.1 Architettura generale

L'applicazione segue un pattern **MVC semplificato**:

- **Model**: collezioni Firestore (users, mezzi, parcheggi, rides, ecc.)
- **View**: interfacce EJS per utenti e gestori
- **Controller**: rotte Express che validano i dati, gestiscono la logica e interagiscono con Firestore o MQTT

2.2 Componenti principali

Componente	Descrizione
Backend REST (Express)	Gestisce logica applicativa, autenticazione, rotte HTTP e interazioni con Firestore
Database Firestore	Collezioni: utenti, mezzi, parcheggi, corse, tariffe, sospensioni
Broker MQTT (Aedes)	Comunicazione tra backend e sottosistema IoT
Emulatore IoT (Hue/Mobishare Emulator)	Simulano spie luminose e comandi blocca/sblocca
Dashboard Gestore (Socket.IO)	Controllo in tempo reale di mezzi e parcheggi
Servizi esterni	Stripe per pagamenti, OpenStreetMap/Nominatim per geolocalizzazione

2.3 Dominio applicativo e modello dati

Il sistema Mobishare é costituito e gestisce le seguenti entità:

- **Utente**: Registrato con email e password, può essere di tipo "utente" o "gestore"
- **Mezzo**: Veicolo della flotta (bicycle muscolare/elettrica, monopattino elettrico)
- **Parcheggio**: Area di prelievo e riconsegna mezzi con coordinate geografiche
- **Corsa**: Sessioni di noleggio con tracciamento temporale e costo
- **Transazione**: Ricariche di credito e pagamenti corse
- **Sistema Fedeltà**: Punti accumulabili e riscattabili in credito
- **Segnalazione**: Danni e guasti rilevati dagli utenti
- **Sospensione**: Blocco temporaneo profilo per saldo negativo

2.4 Architettura logica e pattern MVC

L'applicazione adotta un pattern MVC (Model-View-Controller) semplificato:

Model

- Firestore Collections: Rappresentano le entità di dominio
- Schema-less: Struttura flessibile ma con relazioni logiche tramite campi FK
- Validazione: Gestita a livello controller prima del persisting

View

- Template EJS: Generazione dinamica lato server
- Bootstrap 5: UI responsive e componenti predefiniti
- Client-side: JavaScript vanilla per interazioni real-time

Controller

- Express Routes: Gestione richieste HTTP e business logic
- Middleware: Autenticazione, autorizzazione, validazione
- Service Layer: Astrazione operazioni Firestore e MQTT

2.5 Tecnologie e integrazioni

Backend & Runtime

- Node.js + Express: Server web e gestione sessioni
- Firebase Admin SDK: Autenticazione e accesso sicuro a Firestore
- Aedes MQTT: Broker per comunicazione IoT

Frontend & UI

- EJS Templates: Rendering lato server delle views
- Bootstrap CSS: Framework UI component
- Leaflet + OpenStreetMap: Mappe interattive parcheggi

Servizi Esterni

- Stripe API: Pagamenti sicuri e gestione transazioni
- Nominatim Geocoding: Conversione indirizzi → coordinate
- Socket.IO: Comunicazione real-time dashboard

Emulatori & Testing

- Hue Emulator: Simulazione luci IoT per stati mezzi
- Mobishare Emulator: Dashboard web per controllo visivo
- Arduino ESP8266: Testing hardware connessione MQTT

2.6 Struttura del database (Firestore)

Per la realizzazione dell'applicazione MobiShare è stato adottato Firestore, un database documentale NoSQL. La scelta è stata presa dopo aver analizzato le esigenze del progetto. In un contesto come quello di MobiShare i dati cambiano in modo continuo e simultaneo: utenti che prenotano o terminano corse, mezzi che aggiornano costantemente la posizione e il livello di batteria, operatori che intervengono in caso di anomalie etc etc. In questo scenario, abbiamo ritenuto NoSql una scelta migliore sia per rapidità di aggiornamento che scalabilità e flessibilità (per aggiungere nuove funzionalità molto più facilmente).

Motivazioni della scelta di un database NoSQL

La decisione di utilizzare Firestore al posto di un database relazionale tradizionale (SQL) deriva da alcune esigenze architetturali:

- **Performance in tempo reale:** Firestore permette letture e scritture rapide, permettendo aggiornamenti immediati. Ciò è essenziale in un'applicazione dove la latenza incide sulla qualità del servizio, ad esempio lo sblocco di un veicolo.
- **Flessibilità e adattabilità:** L'app è soggetta a continue evoluzioni (introduzione di sistemi di punti o nuovi tipi di veicoli, ecc.). In un contesto NoSQL, la struttura del database può adattarsi dinamicamente, consentendo di aggiungere o modificare campi nei documenti senza dover eseguire complesse operazioni di adattamento.
- **Gestione delle relazioni**
Pur non disponendo dei meccanismi di JOIN tipici dei database relazionali, Firestore permette di stabilire relazioni logiche tra le entità attraverso l'utilizzo di **riferimenti espliciti** (es. `userId`, `mezzoId`, `id_parcheggio`). Queste connessioni, consentono di mantenere la coerenza dei dati e di ricostruire facilmente i legami tra documenti quando necessario. (simulando appunto le join di un database relazionale)
- **Condivisione**
Essendo il progetto sviluppato e gestito da più persone, si è ritenuto fondamentale disporre di una dashboard o di uno strumento di amministrazione che permettesse a tutti i gestori di interagire in modo simultaneo con il database, garantendo così una gestione coordinata e coerente dei dati.

Struttura dei Documenti

Qui abbiamo fatto una rappresentazione tabellare dei vari documenti (tabelle in sql) del nostro database su firestore.

Documento: User

Contiene i dati anagrafici e di stato degli utenti registrati (sia utenti che gestori).

email	string	email utente
nome	string	nome utente
ruolo	"utente" "gestore"	ruolo dell'utente nel sito
saldo	number	saldo in euro dell'utente
puntiFedelta	number	numero dei punti mobilità accumulati
stato_profilo	"attivo" "sospeso"	stato del profilo dell'utente
createdAt	Timestamp	timestamp creazione account

Documento: Mezzi

Contiene i veicoli della flotta, suddivisi per tipo e stato.

id	number	id del mezzo
tipo	string	
isElettrico	boolean	se il mezzo é assistito o meno
batteria	number null	0-100 null se non ce l'ha
stato	"Disponibile" "In uso" "Non disponibile"	stato disponibilità mezzo
id_parcheggio	number	FK parcheggi
ultimoUtilizzo	Timestamp	

Documento: Parcheggi

Raccoglie le informazioni sui parcheggi disponibili nella città.

id	number	id parcheggio
nome	string	nome del parcheggio

via	string	indirizzo del parcheggio
latitudine	number	coordinate per la mappa
longitudine	number	coordinate per la mappa

Documento: Rides

Memorizza i dati relativi alle corse effettuate dagli utenti.

userId	string	FK users document id
mezzold	number	FK mezzi
mezzoTipo	string	
status	"Attiva" - "in_riepilogo" - "completata"	stato della corsa
startTime	Timestamp	orario di avvio
endTime	Timestamp	orario di fine
costo	number	costo in euro della corsa
durataMinuti	number	durata della corsa in minuti
puntiCorsa	number	punti mobilità guadagnati nella corsa
feedback	string	eventuale feedback utente
danni	string[]	eventuale lista danni
parcheggioPartenza	number	FK parcheggi
parcheggioArrivo	number	FK parcheggi

Documento: danni

Raccoglie le segnalazioni di guasto o anomalie inviate dagli utenti.

mezzold	number	
rideld	string	FK rides document
tipoDanno	string	
segnalatoDa	string	nome utente
dataSegnalazione	Timestamp	timestamp data segnalazione
parcheggio	number	FK parcheggi

Documento: Ricariche

Registra le operazioni di ricarica del credito tramite Stripe.

data	Timestamp	data ricarica
importo	number	importo in euro caricato
metodo	string	metodo di pagamento
paymentIntendId	string	necessario per stripe
status	string	stato transazione
userId	string	FK user document

Documento: Tariffe

Definisce le tariffe per ogni tipologia di mezzo.

costoFisso	number	costo per il noleggio
costoMinuto	number	costo per minuto di utilizzo
mezzo	string	nome mezzo
mezzora	string	costo per 30m di utilizzo

Documento: Shop

Contiene i pacchetti disponibili per il riscatto punti → credito.

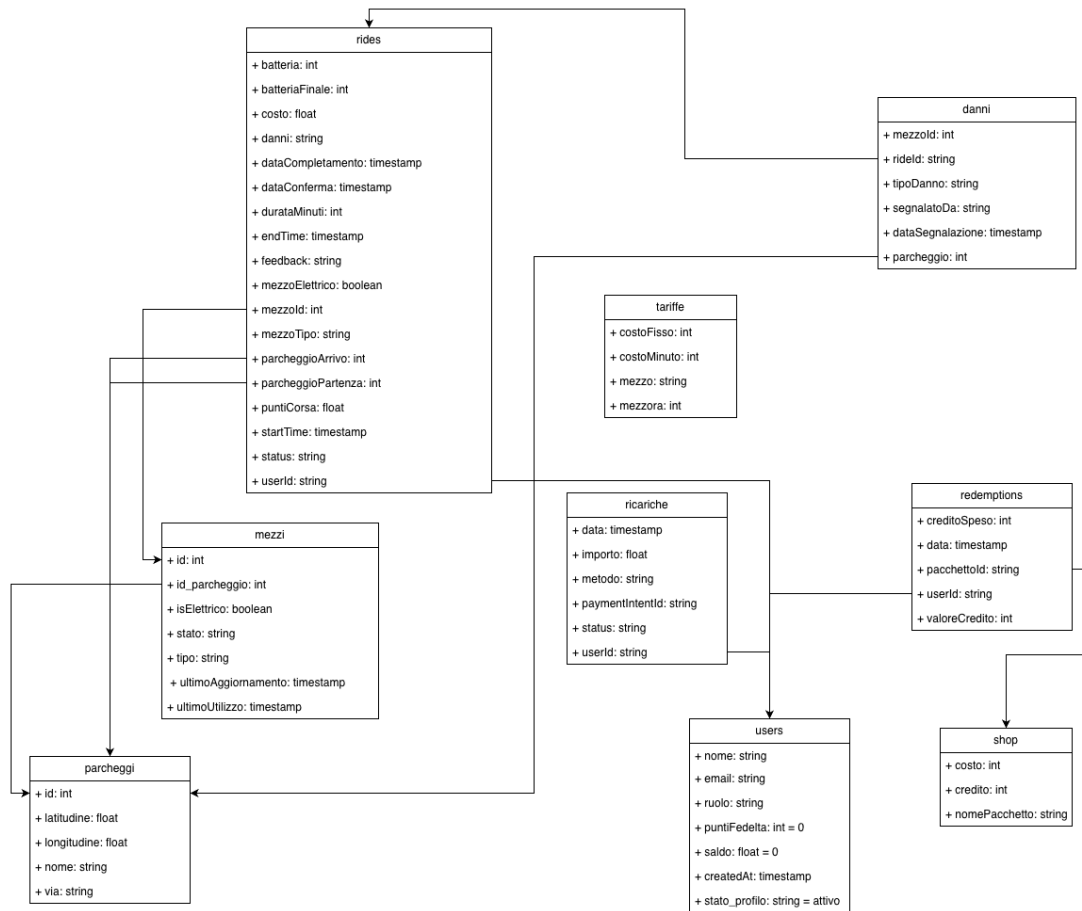
costo	number	costo in euro del pack
credito	number	ricompensa del pack
nomePcchetto	string	nome del pack

Documento: Redemption

Traccia i riscatti di pacchetti dallo shop fedeltà.

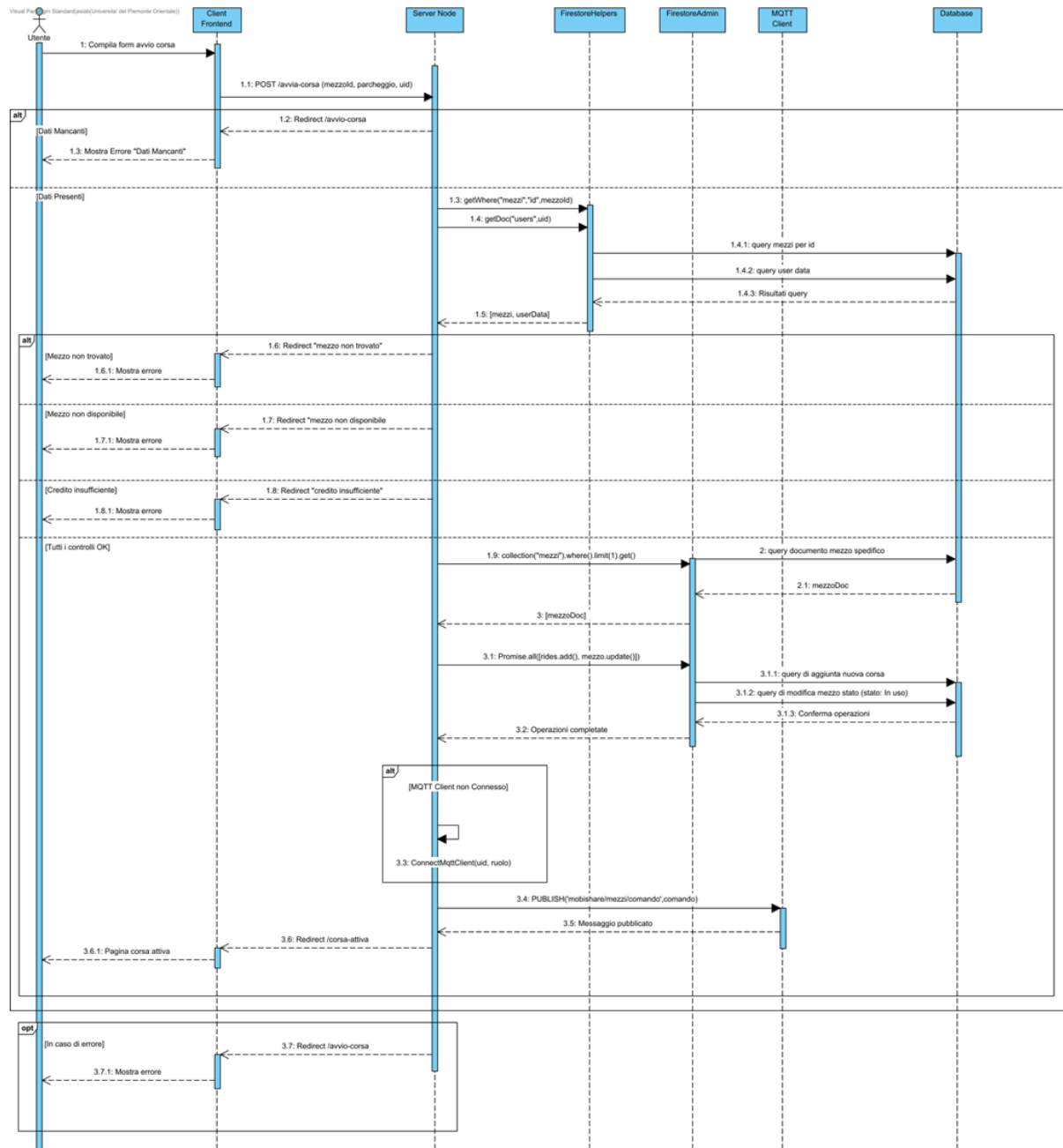
creditoSpeso	number	credito usato per l'acquisto
data	Timestamp	data acquisto
pacchettoid	string	FK pacchetto document
userId	string	FK user document
valoreCredito	number	valore pacchetto

Diagramma UML



2.7 Diagrammi di sequenza

Illustriamo i diagrammi di sequenza di una delle operazioni più complesse: l'inizio di una nuova corsa



3 Comunicazione IoT e protocollo MQTT

Il sistema utilizza il protocollo MQTT per comunicare tra componenti:

Backend/App $\longleftrightarrow$ MQTT Broker (Aedes) $\longleftrightarrow$ Emulatori/Dispositivi

3.1 Funzionamento MQTT

Il protocollo MQTT gestisce la comunicazione con gli elementi (emulati) attraverso un sistema di pubblicazione/sottoscrizione su topic specifici.

```
mqttClient.publish('mobishare/mezzi/comando', {  
  action: 'blocca',  
  mezzoId: 123,  
  userId: 'user123'  
});
```

Esempio Sintassi comando inviato

Gestione comandi dal broker

Il broker MQTT riceve i comandi, li elabora e li invia ai dispositivi sottoscritti ai topic.

```
aedes.on('publish', (packet, client) => {  
  if (packet.topic === 'mobishare/mezzi/comando') {  
    // Invia a tutti gli emulatori sottoscritti  
    broadcastToSubscribers(packet);  
  }  
});
```

Esempio Sintassi broker

3.2 Emulatori

Per rappresentare graficamente gli effetti dei comandi IoT in tempo reale, il sistema utilizza due emulatori autonomi:

Philips Hue Emulator

- Emulatore di riferimento: Basato sull'emulatore mostrato durante le lezioni
- Compatibilità API: Riproduce fedelmente il comportamento del bridge Philips Hue reale
- Funzionalità base: Gestione luci mediante comando
- Limitazioni:
 - Per aggiungere le lampadine bisogna ricaricare il json generato dal server mqtt ogni volta.
 - Non possiamo identificare facilmente i mezzi (dato che mancano i nomi alle lampadine)
 - Richiede il link manuale a ogni avvio

MobiShare Emulator

Sviluppato ad hoc da noi per colmare le limitazioni dell'emulatore Philips integrando il funzionamento dell'mqtt con node, il che permette un'infinità di possibili integrazioni di funzionalità.

- Dashboard web real-time con visualizzazione completa dello stato mezzi
- Controlli manuali integrati per testing diretto
- Indicatori visivi supplementari (livelli batteria, stati complessi e nome mezzi)

3.2 Topic Principali

Comandi e Controllo

- mobishare/mezzi/comando → Comandi blocca/sblocca/in_uso e aggiornamento batteria
- mobishare/mezzi/prenotazione → Richieste prenotazione mezzi

Stati e Aggiornamenti

- mobishare/mezzi/stato → Aggiornamenti stato mezzi (retain)
- mobishare/mezzi/batteria → Aggiornamenti batteria mezzi
- mobishare/mezzi/lista_completa → Sync completo flotta
- mobishare/mezzi/aggiornamenti → Notifiche aggiunta/rimozione

Risposte e Notifiche

- mobishare/mezzi/risposta → Risposte a prenotazioni
- mobishare/mezzi/notifiche → Alert guasti/batteria

Sistema

- system/request-vehicle-list → Richiesta lista mezzi

3.4 Flusso Comunicazione

1. Backend pubblica comandi su mobishare/mezzi/comando
2. Emulatori ricevono e aggiornano stati luci (Hue/Mobishare)
3. Stati aggiornati pubblicati su mobishare/mezzi/stato
4. Dashboard in real-time via Socket.IO per visualizzazione
5. Dispositivi IoT (Arduino) sottoscrivono per aggiornamenti

3.5 Gestione Stati Mezzi Elettrici

L'emulatore gestisce i comandi MQTT e aggiorna lo stato visivo delle luci (verde, arancione e rossa) tramite i vari emulatori per rappresentare graficamente lo stato dei mezzi.

I mezzi elettrici inviano dati relativi alla **batteria**; se il livello scende sotto il 15%, il mezzo diventa automaticamente *non prelevabile* e viene registrato un danno con la descrizione "Batteria scarica".

Batteria < 15%: Mezzo automaticamente "Non disponibile"

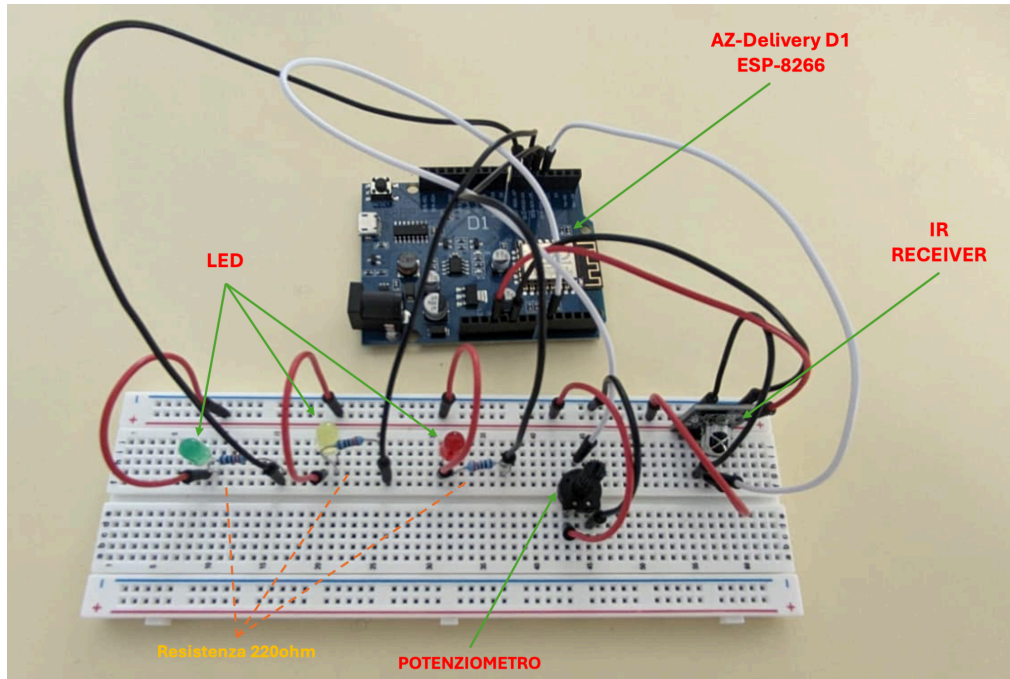
Danno registrato: "Batteria scarica" in collezione danni

Sync automatico: Firestore triggers → aggiornamento MQTT

4.0 Arduino (ESP8266)

4.1 Prototipo Arduino con Comunicazione MQTT

Per dimostrare la funzionalità del sistema in un contesto pseudo-reale, è stato implementato un prototipo basato su Arduino AZ-Delivery D1 (con modulo WiFi ESP8266) che comunica direttamente con il broker MQTT.



4.2 Configurazione Hardware

Il dispositivo Arduino è composto da:

- 3 LED indicatori per la visualizzazione degli stati e della batteria
- 1 ricevitore IR con telecomando per l'interazione
- 1 potenziometro per la simulazione dei parametri

4.3 Funzionalità Implementate

Gestione Connessione e visualizzazione degli stati

- **LED di connessione:** Indicano lo stato della connessione MQTT e WIFI
- **Visualizzazione stati:** Una volta connesso, le lampadine mostrano lo stato del mezzo o il livello della batteria (in base alla modalità)

Simulazione Parametri

- Potenziometro: Simula il consumo di batteria in tempo reale
- Telecomando IR:
 - Selezione della modalità di visualizzazione
 - Scelta del mezzo da controllare
 - Navigazione tra le funzionalità

4.4 Integrazione con il Sistema MQTT

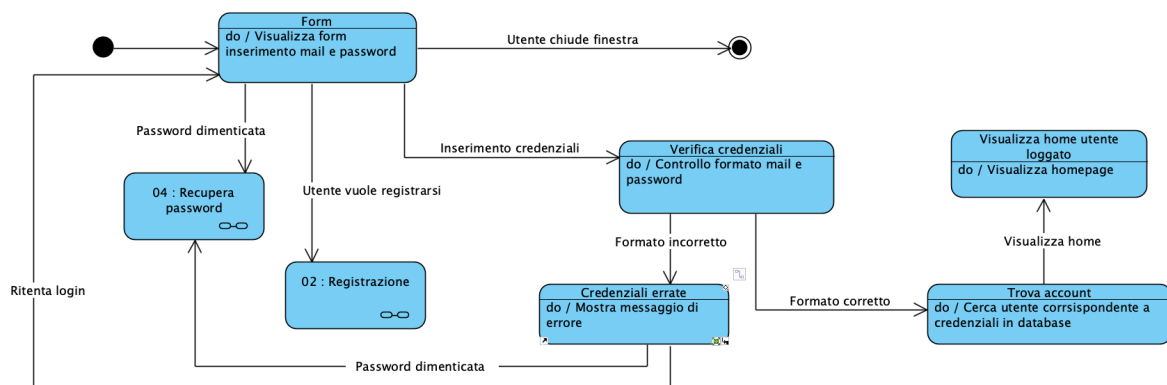
Il prototipo si integra con l'ecosistema MobiShare, sottoscrivendo ai topic MQTT per ricevere comandi e pubblicando aggiornamenti sugli stati dei mezzi.

Queste funzionalità ci permettono di testare in prima persona le varie funzionalità "IOT" senza aver bisogno di emulatori

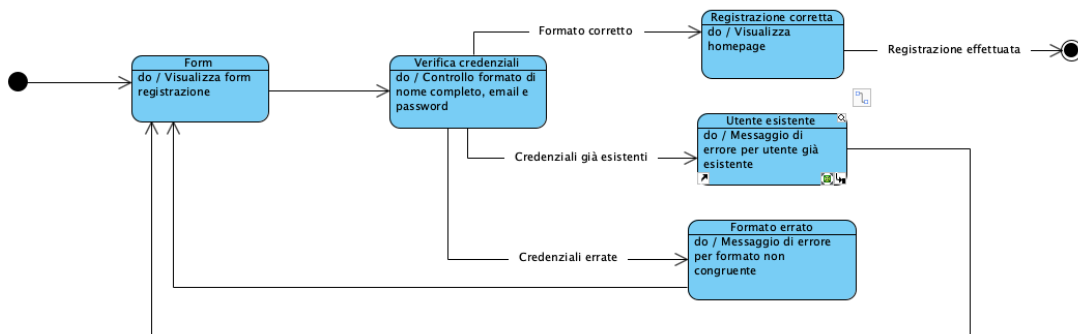
5.0 Diagrammi di stato

In seguito sono illustrati alcuni dei principali diagrammi di stato:

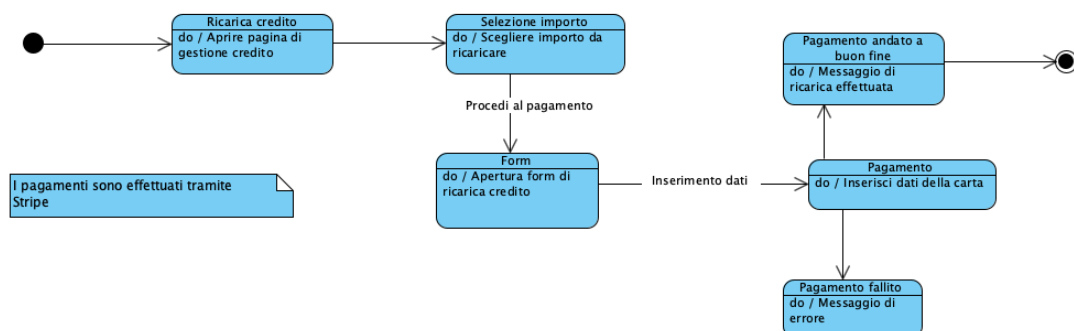
01: Login



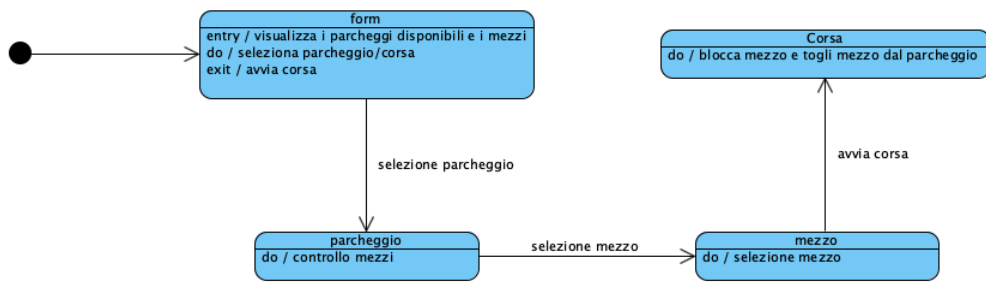
02: Registrazione



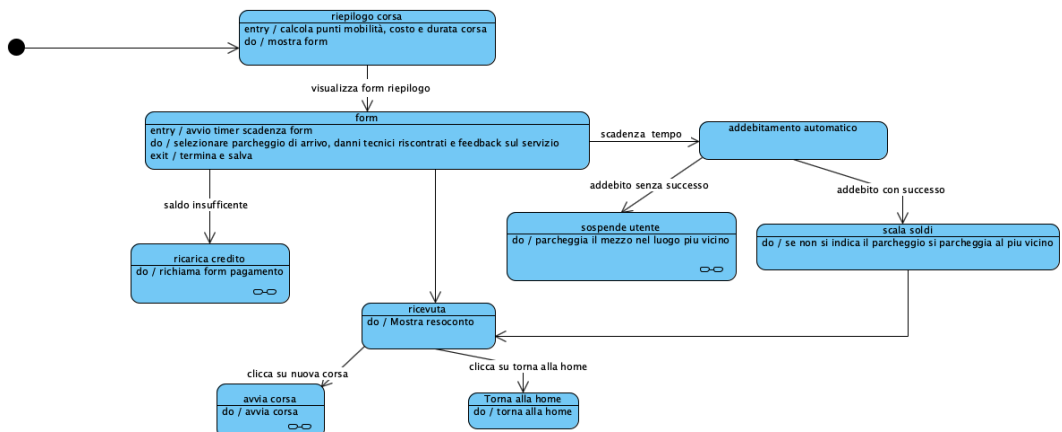
07: Ricarica credito



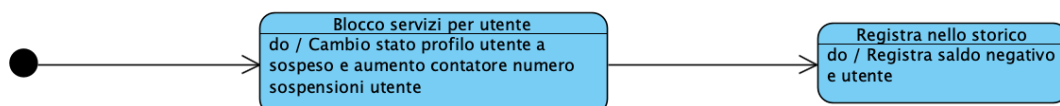
09: Avvia corsa



10: Termina corsa



11: Sospensione utente



29: Gestione parcheggio

